

UNIVERSITÉ D'AIX-MARSEILLE • BUT 2 INFORMATIQUE ARLES

R3.L01 — Représentations et codages des images

TP n°3 : Format JPEG & Utilisation de libjpeg (IJG)

*Guide intégral d'architecture et de mise en œuvre basé sur le cours de **M. Éric Rémy***

Théorie psycho-visuelle, pipeline DCT, interface C/C++, taux de qualité et code intégral

Contrôle TD : 14/10/2026

Contrôle TP : 06/11/2026

Séance TP3 : Format JPEG

Pourquoi JPEG ? La Rupture avec PPM et PGM

Les Limites Infranchissables des Formats Bruts

TP3.TXT & CM

- **Taille excessive du PPM brut** : L'image chat . ppm en 320×240 pixels pèse **226 Ko**. Un cliché 4K pèse 25 Mo !
- **Incompatibilité réseau** : Transmettre ou stocker des images sans compression saturerait les disques et les connexions Web.
- **JPEG (Joint Photographic Experts Group)** : Premier standard mondial de compression d'images photographiques à pertes (norme ISO/IEC 10918-1).

Le POURQUOI : Compression avec Pertes & Système Visuel

- **Modèle psycho-visuel humain (HVS)** : Notre œil ne perçoit pas toutes les informations de manière égale :
 - Très sensible aux variations de luminosité (bâtonnets).
 - Peu sensible aux variations fines de couleur (cônes).
 - Incapable de discerner les très hautes fréquences spatiales dans les zones texturales.
- **Objectif JPEG** : Supprimer **exclusivement** ce que l'œil humain ne peut pas voir !
Résultat : Passage de **226 Ko à 19 Ko** (gain de 91,5 % !).

La Chaîne Complète de Compression JPEG



Synthèse du Pipeline

Les étapes 1, 2, 3, 5 et 6 sont des réorganisations de données ou du codage entropique sans perte (ou presque).

La seule et unique étape de perte d'information irréversible est la Quantification (étape 4), gouvernée directement par le paramètre `quality` (0 à 100) !

L'Espace YCbCr & Le Sous-échantillonnage Chromatique

Décomposition Luminance / Chrominance

$$Y = 0,299 \times R + 0,587 \times G + 0,114 \times B$$

$$Cb = -0,1687 \times R - 0,3313 \times G + 0,5 \times B + 128$$

$$Cr = 0,5 \times R - 0,4187 \times G - 0,0813 \times B + 128$$

- **Y (Luminance)** : Image en niveaux de gris perçue par l'œil.
- **Cb, Cr (Chrominance)** : Écarts de bleu et de rouge par rapport au gris.

Le POURQUOI : Sous-Échantillonnage 4:2:0

Puisque l'œil est presque aveugle aux détails colorés fins :

- **Chroma Subsampling 4:2:0** : On ne conserve qu'un échantillon Cb et Cr pour un bloc de **4 pixels** (division par 2 en largeur et en hauteur).
- **Gain immédiat de 50 %** : Au lieu de 12 valeurs (4 pixels $\times$ 3 canaux), on ne stocke que 4 valeurs $Y + 1 \text{ Cb} + 1 \text{ Cr} = 6$ valeurs !
- **Pourquoi les rectangles sont plus ternes ?** Les bords très vifs des rectangles tracés subissent ce mélange de couleurs 4:2:0, rendant la couleur légèrement adoucie.

Découpage en Blocs 8×8 & Transformation DCT

La DCT 2D (Discrete Cosine Transform)

Chaque composante est découpée en blocs élémentaires de **8 × 8 pixels**.

$$F(u, v) = \frac{1}{4} C(u) C(v) \sum \sum f(x, y) \cos[\dots] \cos[\dots]$$

- **Coefficient DC (0, 0)** : Fréquence nulle. C'est la **valeur moyenne** de luminosité ou couleur du bloc 8×8.
- **63 Coefficients AC** : Fréquences croissantes représentant les variations spatiales et les textures fines.

Le POURQUOI : Pourquoi 8×8 et la DCT ?

- **Concentration d'énergie (Energy Compaction)** : Dans une photo naturelle, les pixels voisins sont très corrélés. La DCT concentre plus de 90 % de l'information dans les premiers coefficients (en haut à gauche).
- **Pourquoi des blocs 8×8 ?** Si le bloc était plus grand (ex: 64×64), le calcul deviendrait trop lourd et les artéfacts de ringing baveraient partout. 8×8 est la taille optimale assurant la stationnarité locale du signal.

La Quantification : Le Cœur de la Réduction de Taille

Division par la Matrice de Quantification Q

Chaque coefficient fréquentiel est divisé par la valeur de la table psycho-visuelle normalisée :

$$F_{\text{quantifié}}(\mathbf{u}, \mathbf{v}) = \text{round}(F(\mathbf{u}, \mathbf{v}) / Q(\mathbf{u}, \mathbf{v}))$$

- Dans la table Q, les valeurs associées aux **hautes fréquences** sont **très grandes** (ex: 50 à 100).
- Conséquence : La quasi-totalité des coefficients de haute fréquence sont **arrondis à 0** !

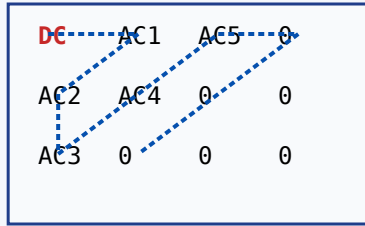
Le POURQUOI : Rôle du Paramètre quality (0 à 100)

```
jpeg_set_quality(&cinfo, quality, TRUE);
```

- Le paramètre quality **redimensionne la matrice Q** :
 - Si quality < 50 : Q est multipliée par un grand facteur → division massive, presque tous les coefficients deviennent 0 (fichier minuscule mais fort effet de blocs).
 - Si quality = 75 (Défaut) : Équilibre parfait.
 - Si quality = 100 : Q est réduite au minimum (1).

Parcours en Zig-Zag & Codage Entropique (Huffman)

Le Parcours en Zig-Zag



Ordonne les coefficients par fréquences croissantes.

Le POURQUOI : Pourquoi regrouper les zéros ?

- **Création d'une longue traînée de zéros** : Grâce au parcours en zig-zag, les coefficients non-nuls sont au début, et tous les zéros sont groupés à la fin.
- **Marqueur EOB (End Of Block)** : Dès que tous les coefficients restants sont nuls, on émet un seul symbole spécial EOB et on passe au bloc suivant !
- **Codage de Huffman sans perte** : Les symboles fréquents reçoivent un code binaire très court (ex: 2 bits), compressant le flux au maximum théorique de Shannon.

L'Intégration de libjpeg : L'Impératif extern "C"

Inclusion des En-têtes C en C++

TP3.TXT Lignes 23-26

```
// Directive obligatoire sur 3 lignes :  
extern "C" {  
#include <jpeglib.h>  
}
```

- Édition de liens sous Linux : drapeau -ljpeg.
- `g++ -Wall -O2 ColorImage.cpp tp3.cpp -ljpeg -o tp3`

Le POURQUOI : Qu'est-ce que le Name Mangling ?

- **En C++ (Name Mangling)** : Pour gérer la surcharge, le compilateur décore les noms de fonctions (ex: `jpeg_create_compress` devient `_Z20jpeg_create_compressP...`).
- **Dans libjpeg (écrite en C pur)** : Les symboles compilés sont enregistrés sous leur nom brut `jpeg_create_compress`.
- **Conséquence sans extern "C"** : Le linker échoue avec l'erreur redoutée : `undefined reference to 'jpeg_create_compress(...)'`.
`extern "C"` ordonne à g++ de respecter la convention C !

Initialisation et Paramétrage

jpeg_example.c

```
struct jpeg_compress_struct cinfo;
struct jpeg_error_mgr jerr;

cinfo.err = jpeg_std_error(&jerr);
jpeg_create_compress(&cinfo);

FILE* outfile = fopen(fname, "wb");
jpeg_stdio_dest(&cinfo, outfile);

cinfo.image_width = width;
cinfo.image_height = height;
cinfo.input_components = 3;
cinfo.in_color_space = JCS_RGB;

jpeg_set_defaults(&cinfo);
jpeg_set_quality(&cinfo, quality, TRUE);
```

Le POURQUOI : Pourquoi FILE* et pas std::ostream ?

- **Héritage historique C** : libjpeg a été conçue pour s'interfacer avec les flux standards C (stdio.h). La fonction jpeg_stdio_dest prend obligatoirement un pointeur FILE*.
- **Importance du mode binaire "wb"** : Sous certains systèmes (Windows), omettre le "b" corrompt le fichier en convertissant les octets 0x0A en 0x0D 0x0A !
- **Ordre des appels** : Toujours appeler jpeg_set_defaults() APRÈS avoir défini in_color_space = JCS_RGB, car les tables par défaut dépendent de l'espace source.

Compression Ligne par Ligne (Scanlines)

Boucle de Compression

ColorImage::writeJPEG

```
jpeg_start_compress(&cinfo, TRUE);  
  
JSAMPROW row_pointer[1];  
int row_stride = width * 3;  
const JSAMPLE* buf = (const JSAMPLE*)array;  
  
while (cinfo.next_scanline < cinfo.image_height) {  
    row_pointer[0] = (JSAMPROW)&buf[cinfo.next_scanline * row_stride];  
    jpeg_write_scanlines(&cinfo, row_pointer, 1);  
}  
  
jpeg_finish_compress(&cinfo);  
fclose(outfile);  
jpeg_destroy_compress(&cinfo);
```

Le POURQUOI : Compatibilité Parfaite avec notre RAM

- **Aucune allocation temporaire requise** : Comme vu au TP2, notre tableau `Color* array` stocke en continu `R, G, B, R, G, B, \dots`. C'est **exactement** ce qu'attend `libjpeg` !
- **Le compteur interne `cinfo.next_scanline`** : Géré automatiquement par la bibliothèque, il s'incrémente à chaque scanline écrite.
- **Nettoyage obligatoire** : `jpeg_destroy_compress` libère les mémoires de travail internes de la bibliothèque.

Analyse Expérimentale : Qualité vs Taille (21 Images)

Fichier Produit	Qualité	Taille sur Disque	Taux de Compression	Observation Visuelle
chat_rectangles.ppm	Brut (Sans)	226 Ko (230 458 o)	1:1 (Référence)	Couleurs éclatantes, aucun défaut
out_000.jpg	0 %	2,9 Ko	78 : 1	Effet de blocs extrême, bouillie de pixels
out_020.jpg	20 %	8,5 Ko	26 : 1	Contours bruités (ringing), flou visible
chat.jpg / out_075.jpg	75 % (Défaut)	19 Ko	12 : 1	Excellente netteté , défauts invisibles à l'œil
out_100.jpg	100 %	73 Ko	3,1 : 1	Qualité maximale mais perte persistante sur les bords vifs

Conclusions fondamentales demandées par M. Rémy

1. chat.jpg (appel sans paramètre de qualité) et out_075.jpg font **exactement la même taille (19 Ko)** : la qualité par défaut de libjpeg est bien de **75 %**.
2. Même à 100 %, les rectangles rouges, verts et bleus sont **plus ternes qu'en PPM** à cause du sous-échantillonnage chromatique YCbCr 4:2:0 et de l'adoucissement des transitions abruptes par la DCT.

Bonus : Décompression JPEG (ColorImage::readJPEG)

Mécanisme de Décompression

Bonus TP3

```
struct jpeg_decompress_struct cinfo;
struct jpeg_error_mgr jerr;

cinfo.err = jpeg_std_error(&jerr);
jpeg_create_decompress(&cinfo);
jpeg_stdio_src(&cinfo, infile);

jpeg_read_header(&cinfo, TRUE);
cinfo.out_color_space = JCS_RGB; // Force le rendu RGB
jpeg_start_decompress(&cinfo);

ColorImage* res = new ColorImage(cinfo.output_width,
                                cinfo.output_height);
JSAMPLE* buf = (JSAMPLE*)res->array;
JSAMPROW row_pointer[1];

while (cinfo.output_scanline < cinfo.output_height) {
    row_pointer[0] = &buf[cinfo.output_scanline * width * 3];
    jpeg_read_scanlines(&cinfo, row_pointer, 1);
}
```

Le POURQUOI : Lecture Asynchrone de l'En-tête

- **Récupération des dimensions** : Contrairement au PPM où on lit la largeur et la hauteur en texte dans l'en-tête, JPEG stocke ces valeurs dans les marqueurs SOF (Start of Frame). C'est `jpeg_read_header` qui les extrait dans `cinfo.output_width` et `output_height`.
- **Allocation exacte** : On alloue l'image dynamiquement en RAM uniquement après avoir lu ces dimensions réelles.

Synthèse pour le Contrôle Théorique & Pratique

Questions types du Contrôle Théorique (14/10)

- **Quelle étape de JPEG est avec perte ?** Uniquement la **quantification** (division par Q). La DCT et Huffman sont mathématiquement réversibles.
- **Pourquoi convertir en YCbCr ?** Pour exploiter la faible sensibilité de l'œil aux couleurs en sous-échantillonnant Cb et Cr (gain immédiat de 50%).
- **Pourquoi le parcours en zig-zag ?** Pour regrouper les zéros à la fin du bloc et émettre un seul marqueur EOB.

Points clés du Contrôle Pratique TP (06/11)

- **Liaison de la bibliothèque** : Toujours ajouter le drapeau `-ljpeg` lors de la compilation `g++`.
- **Inclusion C en C++** : Encadrer impérativement l'inclusion de `jpeglib.h` par `extern "C" { ... }`.
- **Gestion rigoureuse des fichiers** : Toujours ouvrir en `"wb"` ou `"rb"` et fermer avec `fclose()`.

Annexe Code (1/4) : ColorImage.hpp (Intégral TP3)

Fichier d'en-tête ColorImage.hpp enrichi pour JPEG

Header C++

```
#ifndef COLORIMAGE_HPP
#define COLORIMAGE_HPP
#include <stdint>
#include <iostream>
#include <stdexcept>
#include <string>

class Color {
public:
    uint8_t r, g, b;
    inline Color(uint8_t _r = 0, uint8_t _g = 0, uint8_t _b = 0) : r(_r), g(_g), b(_b) {}
    friend bool operator==(const Color& c1, const Color& c2) { return c1.r == c2.r && c1.g == c2.g && c1.b == c2.b; }
    friend Color operator*(double a, const Color& c) { return Color((uint8_t)(a * c.r), (uint8_t)(a * c.g), (uint8_t)(a * c.b)); }
    friend Color operator+(const Color& c1, const Color& c2) { return Color(c1.r + c2.r, c1.g + c2.g, c1.b + c2.b); }
};

class ColorImage {
private:
    const uint16_t width, height;
    Color* array;
    ColorImage() = delete;
public:
    ColorImage& operator=(const ColorImage& b) = delete;
    ColorImage(uint16_t w, uint16_t h);
    ColorImage(const ColorImage& orig);
    ~ColorImage();
    inline const uint16_t& getWidth() const { return width; }
    inline const uint16_t& getHeight() const { return height; }
    inline Color& pixel(uint16_t x, uint16_t y) { return array[y * width + x]; }
    inline const Color& pixel(uint16_t x, uint16_t y) const { return array[y * width + x]; }
    void clear(Color c = Color());
    void rectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, Color c);
    void fillRectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, Color c);
    void writePPM(std::ostream& os) const;
    static ColorImage* readPPM(std::istream& is);
    ColorImage* simpleScale(uint16_t w, uint16_t h) const;
    ColorImage* bilinearScale(uint16_t w, uint16_t h) const;

    // Nouveautés TP3
    void writeJPEG(const char* fname, unsigned int quality = 75) const;
    static ColorImage* readJPEG(const char* fname);
};
#endif
```

Annexe Code (2/4) : ColorImage::writeJPEG

ColorImage.cpp : Implantation complète de la compression JPEG

Méthode writeJPEG

```
extern "C" {
#include <jpeglib.h>
}

void ColorImage::writeJPEG(const char* fname, unsigned int quality) const {
    if (!fname) throw std::invalid_argument("Nom de fichier nul");
    struct jpeg_compress_struct cinfo;
    struct jpeg_error_mgr jerr;
    cinfo.err = jpeg_std_error(&jerr);
    jpeg_create_compress(&cinfo);

    FILE* outfile = fopen(fname, "wb");
    if (!outfile) { jpeg_destroy_compress(&cinfo); throw std::runtime_error("Impossible d'ouvrir le fichier"); }
    jpeg_stdio_dest(&cinfo, outfile);

    cinfo.image_width = width;
    cinfo.image_height = height;
    cinfo.input_components = 3;
    cinfo.in_color_space = JCS_RGB;
    jpeg_set_defaults(&cinfo);
    jpeg_set_quality(&cinfo, quality, TRUE);

    jpeg_start_compress(&cinfo, TRUE);
    JSAMPROW row_pointer[1];
    int row_stride = width * 3;
    const JSAMPLE* buffer = (const JSAMPLE*)array;

    while (cinfo.next_scanline < cinfo.image_height) {
        row_pointer[0] = (JSAMPROW)&buffer[cinfo.next_scanline * row_stride];
        jpeg_write_scanlines(&cinfo, row_pointer, 1);
    }

    jpeg_finish_compress(&cinfo);
    fclose(outfile);
    jpeg_destroy_compress(&cinfo);
}
```

Annexe Code (3/4) : ColorImage::readJPEG

ColorImage.cpp : Implantation complète de la décompression JPEG

Méthode readJPEG

```
ColorImage* ColorImage::readJPEG(const char* fname) {
    if (!fname) throw std::invalid_argument("Nom de fichier nul");
    FILE* infile = fopen(fname, "rb");
    if (!infile) throw std::runtime_error("Impossible d'ouvrir le fichier");

    struct jpeg_decompress_struct cinfo;
    struct jpeg_error_mgr jerr;
    cinfo.err = jpeg_std_error(&jerr);
    jpeg_create_decompress(&cinfo);
    jpeg_stdio_src(&cinfo, infile);

    jpeg_read_header(&cinfo, TRUE);
    cinfo.out_color_space = JCS_RGB; // Forcer la sortie en RGB 24 bits
    jpeg_start_decompress(&cinfo);

    ColorImage* res = new ColorImage(cinfo.output_width, cinfo.output_height);
    int row_stride = cinfo.output_width * cinfo.output_components;
    JSAMPLE* buffer = (JSAMPLE*)res->array;
    JSAMPROW row_pointer[1];

    while (cinfo.output_scanline < cinfo.output_height) {
        row_pointer[0] = &buffer[cinfo.output_scanline * row_stride];
        jpeg_read_scanlines(&cinfo, row_pointer, 1);
    }

    jpeg_finish_decompress(&cinfo);
    fclose(infile);
    jpeg_destroy_decompress(&cinfo);
    return res;
}
```

Annexe Code (4/4) : Programme de Test tp3.cpp

tp3.cpp : Validation complète du TP3

Programme Principal

```
#include "ColorImage.hpp"
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>

int main() {
    try {
        // 1. Chargement et tracé des rectangles
        std::ifstream fIn("chat.ppm", std::ios::binary);
        ColorImage* im = ColorImage::readPPM(fIn);
        im->rectangle(10, 10, im->getWidth() - 20, im->getHeight() - 20, Color(255, 0, 0));
        im->rectangle(14, 14, im->getWidth() - 28, im->getHeight() - 28, Color(0, 255, 0));
        im->rectangle(18, 18, im->getWidth() - 36, im->getHeight() - 36, Color(0, 0, 255));

        // 2. Sauvegarde JPEG par défaut (qualité 75%) : 226 Ko -> 19 Ko
        im->writeJPEG("chat.jpg");

        // 3. Boucle des 21 taux de qualité (0 à 100 de 5 en 5)
        for (unsigned int q = 0; q <= 100; q += 5) {
            std::ostringstream oss;
            oss << "out_" << std::setfill('0') << std::setw(3) << q << ".jpg";
            im->writeJPEG(oss.str().c_str(), q);
        }

        // 4. Test Bonus : Décompression JPEG
        ColorImage* test = ColorImage::readJPEG("out_075.jpg");
        delete test;
        delete im;
        std::cout << "TP3 VALIDE AVEC SUCCES !" << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "ERREUR : " << e.what() << std::endl; return 1;
    }
    return 0;
}
```